Veri-Store Security Model

Mac Payton & Albert Huynh

May 3, 2026
Version 0.1.2

Contents

1 Introduction

1.1 Document Purpose

This document will be used to identify threats, design mitigations, and verify the security properties of the Veri-Store system throughout the development lifecycle. It will be updated as components are implemented and new threats are discovered.

1.2 System Overview

Veri-Store is a distributed file storage system built using 3-of-5 Reed-Solomon erasure coding. It splits a single data block into 5 fragments in such a way that the original block of data can be built with only 3 of the original fragments.

The main security concerns are (a) ensuring integrity of the fragments and (b) ensuring that a fragment belongs to the data block being reconstructed. If a fragment has been tampered with (integrity) or was not part of the original block (authenticity), but data block construction goes forward anyway, an adversary could inject malicious code into the system.

Veri-Store addresses these concerns using homomorphic fingerprinting, allowing each fragment to be indepently verified on their respective servers as belonging to the correct data block without needing to reconstruct the full block.

2 Scoping

2.1 In Scope

This security model focuses on the core veri-store protocol and implementation. In scope are the parts of the system that directly handle data fragments, fingerprints, and cross-checksums:

- Homomorphic fingerprint generation and verification for data blocks and fragments.
- Reed–Solomon erasure encoding and decoding (3-of-5 scheme) used to split and reconstruct data.
- Client logic that disperses data, generates the fingerprinted cross-checksum (fpcc), and verifies fragments on retrieval.
- Storage server API and logic for receiving, storing, verifying, and deleting fragments.
- The fingerprinted cross-checksum protocol, including how servers use it to check fragment integrity.
- Interactions between the veri-store client and storage servers over HTTP (FastAPI and `httpx`).
- Protection of data integrity and authenticity for stored fragments and reconstructed blocks.

2.2 Out of Scope

Some important security topics are intentionally out of scope for this project. We document them here so that readers do not assume they are handled:

- Physical security of machines (e.g., data center access control, hardware tampering).
- Operating system and hypervisor security, including kernel exploits and container escapes.
- Network-layer protections such as DDoS mitigation, BGP attacks, or link encryption; we assume a working TCP/IP network and optionally TLS.
- User authentication and authorization beyond the "authenticated client" abstraction; account management and access control are not modeled.
- Confidentiality of stored data; veri-store detects tampering but does not encrypt data. Sensitive data must be encrypted before use.
- Availability in the face of more than two failed or unreachable servers (beyond the 3-of-5 erasure code tolerance).
- Advanced Byzantine behavior such as large-scale collusion between many servers or compromised clients coordinating attacks.

3 Use Scenarios

We expect a number of possible use cases of the Veri-Store system, given in the non-exhaustive list below:

- Intranet for an organization, providing remote access to documents and files to organization members.
- Academic research and testing for erasure coding systems.
- Static asset distribution across geographically dispersed servers.
- Medical records storage, providing both fault tolerance and tamper detection needed for legal compliance.
- Archival storage for organizations that need redundancy without the cost of full replication.

3.1 Primary Use Case

Users will access Veri-Store through a client application, using it to store files which can be accessed later from the network. The client application can store data, retrieve files, verify integrity, and reconstruct the original data block. We will start with a single-user application (akin to a flash drive over the network), with hopes to move Veri-Store to a multi-user system where files can be shared.

The servers in this use case are to be semi-trusted - we assume that adversaries will attempt to tamper with data fragments if they are able to break into the system. (See Section 6 for a formal discussion of trust levels.) Although a network environment (both LAN and public internet) is the goal, for the sake of this project, we will be hosting each "server" on a single machine through different `localhost` ports.

3.2 Anti-Scenarios

The following use cases are unsupported and/or known to be insecure:

- Running servers on machines which are wholly untrusted or known to be compromised.
- Applications with strict latency requirements. Erasure coding and fragment verification introduce non-trivial computational overhead.
- Unencrypted sensitive data - Veri-Store is not an encryption scheme. Sensitive data should be encrypted before being given to Veri-Store.
- We cannot guarantee availability if more than 2 of the 5 servers are unavailable.

4 Dependencies

4.1 External Libraries

Mathematical operations and erasure coding

- `galois [≥ v0.3.8]`: Finite field arithmetic over $\mathbb{F}_{256}$, primarily utilized in developing Cauchy matrices
- `numpy [≥ v1.26.0]`: Array operations and polynomial manipulation
- `reedsolo [≥ v1.7.0]` (*optional*): Encoding and decoding data before dispersal

Web framework

- `fastapi [≥ v0.110.0]`: REST API for client-server communication
- `uvicorn [≥ v0.29.0]`: ASGI server for hosting the API

HTTP client

- `httpx [≥ v0.27.0]`: For inter-server communication and fragment distribution

API data validation

- `pydantic [≥ v2.6.0]`: Data validation and parsing for API requests/responses

Testing

- `pytest [≥ v8.0.0]`: For running unit and integration tests
  - `pytest-asyncio [≥ v0.23.0]`: For testing asynchronous code paths

4.2 System Dependencies

Veri-store also depends on the underlying platform. We treat these as given and do not try to secure them ourselves:

- **Network stack**: We assume a working TCP/IP stack that can deliver HTTP requests between clients and servers. Attacks on routing or congestion control are out of scope.
- **File system**: We rely on the local file system to store fragment files and metadata. We assume basic guarantees such as "writes that complete are durable" and that file permissions are configured correctly by the operator.

- **Operating system**: We assume the OS provides process isolation, memory protection, and a reasonably bug-free implementation of standard syscalls. Kernel-level exploits and privilege escalation are not modeled here.

5 Implementation Assumptions

- At most f out of n total servers may be compromised or fail and the system will still maintain data integrity
- Network communication channels are/will be authenticated
- Storage servers have sufficient disk space for fragments of size $1/m$ of original block
- Homomorphic fingerprint collision probability is negligible
- Erasure coding library correctly implements Reed-Solomon

6 Trust Levels

6.1 Administrator

System administrators have full access to the servers hosting the veri-store system and the stored data. They are responsible for maintaining the infrastructure, applying updates, and ensuring the system is running smoothly. The security model assumes that administrators are trusted and will not intentionally compromise data integrity or confidentiality. However, they have the capability to access stored fragments and fingerprints, so it is important to consider this trust level when designing mitigations for information disclosure threats.

Example System Administrators:

- Developers (e.g., Albert and Mac) during development and testing
- IT staff responsible for server/data maintenance in a production deployment
- Cloud provider administrators if deployed on cloud infrastructure (e.g., AWS, Azure)
- System operators with root access to physical or virtual servers
- *Additional examples to be added*

6.2 Authenticated Client

The term "authenticated client" refers both to the end users and the application they use to access the Veri-Store system.

An authenticated client has verified their identity to the system and is authorized to store and retrieve their own data. They are assumed to be legitimate, but may make mistakes or run buggy software which can violate system security. They should not have access to other users' data or have the ability to violate system integrity.

6.3 Storage Server

Storage servers are *semi-trusted*. Each server is expected to run the published veri-store code and follow the protocol, but we do not assume that every server is always honest or uncompromised. A storage server may:

- Crash or become unavailable.
- Return stale, corrupted, or tampered fragments.
- Attempt to forge data that passes simple hash checks.

The security model assumes that not all servers are malicious at the same time. In the 3-of-5 configuration, we rely on the homomorphic fingerprinting and cross-checksum scheme to detect inconsistent or forged

fragments from a minority of servers. If more than two servers are compromised in a coordinated way, the integrity guarantees can fail. Servers do not have permission to modify the fpcc; they only store it and use it for verification. They also do not see user identities directly; they only see block identifiers and fragment indices.

6.4 Untrusted Network

We assume a TLS connection between clients and servers, although we do not rely on it for data integrity. Adversaries will be able to observe network traffic, and it is assumed that they will, in theory, be able to inject network traffic.

However, the homomorphic fingerprinting scheme allows for any inconsistencies between data fragments to be caught before reconstruction. Any tampering will be detected because the tampered fragment's fingerprint won't match the expected encoding of the other fragments' fingerprints. In order to not be caught, an adversary would need to provide a consistent set of tampered fragments which all pass verification. This is computationally infeasible given the collision resistance of the fingerprinting scheme.

Unfortunately, given the material constraints of this project (i.e., "servers" existing on a single machine as different `localhost` ports), we are unable to fully test this.

7 Entry Points

7.1 Client API

The client API is the main way that users interact with veri-store. It groups three high-level operations:

- **Store operation**: The client sends a data block to veri-store. The client encodes the block into fragments, generates the fpcc, and then issues HTTP requests to store each fragment on its assigned server.
- **Retrieve operation**: The client requests a stored block by its identifier. It asks multiple servers for fragments, verifies each fragment against the fpcc, and reconstructs the original data once enough valid fragments are available.
- **Verify operation**: The client (or an operator) can re-check that stored fragments are still consistent with the fpcc, without needing to reconstruct the full block. This detects tampering after the initial store.

7.2 Inter-Server Communication

In the basic project deployment, all "servers" run on the same host, but the design assumes they could be separate machines on a network. Inter-server communication is limited and well-defined:

- **Fingerprint exchange**: In extended deployments, servers may exchange fingerprint values or fpcc digests to support cross-checking and audit workflows. This path is treated as semi-trusted and is protected by the same fingerprinting assumptions as the client API.
- **Fragment retrieval**: A server may request fragments from peer servers to assist with recovery or migration. Any fragment obtained this way is still subject to the same fpcc-based verification before being used.

7.3 Storage Subsystem

The storage subsystem covers all local file operations used to persist fragments and metadata on each server:

- **Fragment write**: The server writes encoded fragments and their associated metadata (including fpcc digests and verification status) to disk. Writes use a write-then-rename pattern to reduce the chance of partial files.
- **Fragment read**: The server reads fragments and metadata back from disk to answer client requests or to perform re-verification. If a file is missing or malformed, the server treats the fragment as unavailable.
- **Fingerprint storage**: The server stores enough information (such as fpcc digests or JSON-serialized fpcc) to allow future integrity checks without needing the client to resend the fpcc.

8 Protected Assets

For the below items, we will examine:

- Trust levels
- Confidentiality
- Integrity
- Availability

8.1 Data Blocks

8.1.1 Trust Levels

The original data blocks shall only be accessible to

a) the user who uploaded it and
b) server administrators.

8.1.2 Confidentiality

In a production deployment, we would not want to trust the server – the fragments would be encrypted in such a way that the server would not be able to read the underlying data. However, such encryption is beyond the scope of the current implementation.

Our current implementation only accounts for a single user, so access control to data blocks is trivial. In a multi-user system, though, we would implement user IDs (stored as metadata with each fragment) and access tokens provided to users at login. The server confirms that the provided access token is associated with the user ID which has permission to access the data block.

8.1.3 Integrity

The original data block uploaded by a user must be the same data returned to a user upon request, enforced through the use of homomorphic fingerprints. See section 8.3 for more details.

8.1.4 Availability

Availability of a given data block is dependent on m out of n valid fragments being available. In the current implementation, $m = 3$ and $n = 5$.

8.2 Erasure-Coded Fragments

8.2.1 Trust Levels

The erasure-coded fragments will only be accessible to server administrators, and accessible to clients only for the purpose of verification and reconstruction.

8.2.2 Confidentiality

Clients will be able to access erasure-coded fragments for the purpose of validating and rebuilding the original data block. However, they shall not be able to access fragments of data blocks which they did not upload. (See section 8.1.2, para. 2 for a fuller discussion.)

As above, in a production deployment, we do not want server administrators to be able to access the data stored in a fragment; however, the encryption is beyond the scope of our current implementation.

8.2.3 Integrity

Servers should not be able to change any part of the data fragment. Otherwise, the original data block cannot be reconstructed.

Unfortunately, the integrity of individual fragments cannot be guaranteed. A compromised server can still modify a stored fragment, although any such tampering will be detected by the homomorphic fingerprinting scheme before the fragment is used for reconstruction. This prevents corrupted (and possibly malicious) data from being included in the reconstructed block.

8.2.4 Availability

At least m fragments must be available to reconstruct the original data block. For our implementation, that means $n - m = 2$ fragments can be unavailable and the original data block can still be reconstructed.

8.3 Homomorphic Fingerprints

8.3.1 Trust Levels

In our current implementation, server administrators have access to view and possibly modify the stored fingerprints. Clients have access to view fingerprints for the purposes of fragment verification.

8.3.2 Confidentiality

In a production deployment, fingerprints should be encrypted while on the server to prevent such tampering. However, such encryption is out of the scope of our current implementation.

While fingerprints themselves do not reveal any information, an adversary who collects them can confirm if suspected data is stored in a fragment.

8.3.3 Integrity

If the underlying fragment is tampered with, the resulting fingerprint will be changed. This change will be detected during the verification process and the associated fragment not used for block reconstruction.

However, if an adversary can modify both the fragment and the stored fingerprint in a consistent manner, the tampering would go undetected. The collision resistance of the fingerprinting scheme makes forging a consistent fingerprint computationally infeasible. This is a known limitation and is not fully mitigated in our implementation.

8.3.4 Availability

Fingerprints are calculated from the fragments and occur as two types:

1. the fingerprints stored alongside the fragments at upload
2. and the fingerprint calculated from a fragment during the verification process

For the first: so long as the `fpcc` is available, the stored fingerprints will be available. For the second, so long as the fragment is available, the fingerprint can be calculated.

8.4 System Resources

8.4.1 Trust Levels

Server administrators have full access to system resources. Authenticated clients consume resources proportional to their requests (e.g., storage, retrieval, and verification operations).

8.4.2 Confidentiality

Resource usage metrics (e.g., CPU, memory, disk, bandwidth) should not be exposed to clients, as this information could be used to infer system capacity or identify optimal timing for DoS attacks.

8.4.3 Integrity

Resource consumption should be proportional to legitimate usage. A client should not be able to trigger disproportionate resource consumption through malformed or malicious requests — for example, sending a request that causes the server to perform unbounded computation or allocate unbounded memory.

8.4.4 Availability

System resources must remain available to service legitimate requests. The primary threat is denial of service — an adversary exhausting CPU, memory, disk space, or network bandwidth to prevent legitimate clients from storing or retrieving data.

In our current implementation, client-side retry limits (a maximum of 3 attempts) and linear backoff (delay increasing by 0.5 seconds every attempt) are in place, which partially mitigate resource exhaustion from repeated failed requests. However, rate limiting and request quotas are not fully implemented, leaving the system partially vulnerable to DoS attacks from high request volumes.

9 Data Flow Diagram

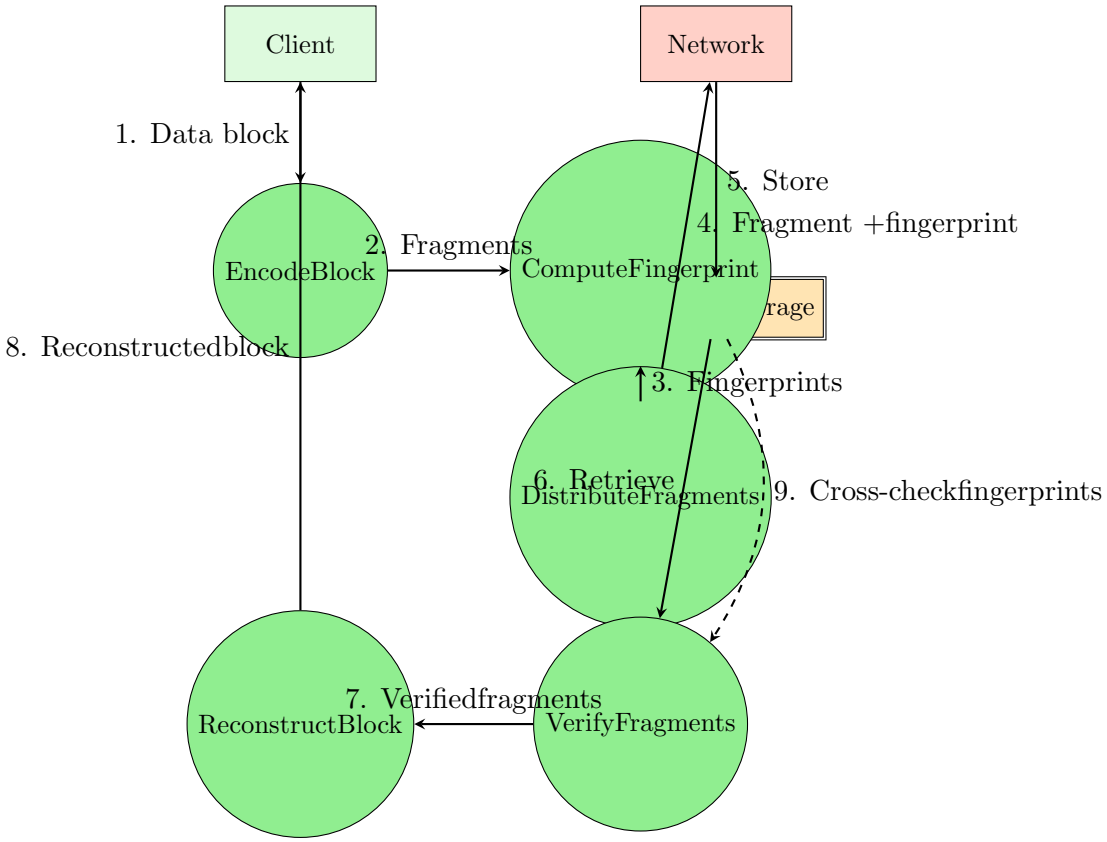

Figure 1: High-level data flow diagram for veri-store store and retrieve operations.

9.1 DFD Component Descriptions

9.1.1 Client

The Client is the entry-point process that initiates store (PUT) and retrieve (GET) operations. It is trusted to run the veri-store protocol correctly, but it is still treated as a software component that can fail due to bugs or operator error. For writes, the Client invokes encoding, computes the fingerprinted cross-checksum (fpcc), and dispatches fragments to storage servers. For reads, it gathers candidate fragments, verifies them locally, and only then reconstructs the original block. This placement makes the Client the final integrity decision point.

9.1.2 Encode Block

Encode Block is a trusted process that converts an input data block into n Reed-Solomon fragments with threshold m (for this project, typically 3-of-5). The process pads input, computes coded fragments, and tags each fragment with metadata (index, block id, n, m, and original length). Its security role is correctness: if encoding is wrong, availability and integrity both degrade because reconstruction may fail or produce incorrect bytes.

9.1.3 Compute Fingerprint

Compute Fingerprint is a trusted process that builds the fpcc structure used for integrity verification. It computes per-fragment hash commitments and homomorphic fingerprints (for indices below m), and derives challenge point r via the random-oracle construction from fragment hashes. The output fpcc binds fragments to a single logical block and enables later detection of tampered, stale, or mismatched fragment responses.

9.1.4 Distribute Fragments

Distribute Fragments is a trusted client-side networking process that sends each fragment and the shared fpcc to its designated server endpoint. It runs requests in parallel, applies timeout bounds, and treats server failures as expected partial-fault behavior. The process succeeds only if at least m servers acknowledge storage, preserving the threshold availability assumption.

9.1.5 Network

Network is explicitly untrusted. An adversary may delay, drop, replay, reorder, or modify packets, and may make individual servers appear unavailable. The design therefore does not rely on network honesty for integrity: retrieved data is always re-verified against fpcc before decode. Timeouts and best-effort fan-out/fan-in behavior are used so the client can tolerate partial outages.

9.1.6 Fragment Storage

Fragment Storage represents server-local persistent stores that hold fragment bytes and associated metadata. It is semi-trusted for durability but not for integrity: servers can crash, lose state, or return corrupted values. The threat model assumes these failures are detectable by client-side verification and threshold reconstruction, rather than prevented by fully trusted storage nodes.

9.1.7 Verify Fragments

Verify Fragments is a trusted process that checks each retrieved fragment against fpcc before it is eligible for decode. Verification first checks the fragment hash commitment and then (for indices below m) checks the homomorphic fingerprint equation. Any fragment that fails consistency checks, has malformed encoding, or disagrees on fpcc context is rejected. This step provides integrity filtering under Byzantine or unreliable server behavior.

9.1.8 Reconstruct Block

Reconstruct Block is a trusted decoding process that rebuilds the original data from any m verified fragments using Reed-Solomon decode. It runs only after the verification phase has accepted enough fragments, then removes padding to recover the original byte length. Security-wise, this enforces a strict pipeline: no unverified fragment is allowed to influence reconstructed output.

10 Threat Analysis: STRIDE

Security Guarantees Achieved

The homomorphic fingerprinting scheme implemented in `src/verification/` provides the following formal guarantees that bound the threats analyzed in this section.

- **Detection Rate** The SHA-256 hash check catches any byte-level modification with miss probability 2^{-256}. For the first m fragments, the GF(256) fingerprint check provides a second layer with a theoretical miss bound of at most 1/256 per check (theorem 3.4 of Hendrics et al.). In practice, empirical testing over all byte positions and flip masks confirms a 100% detection rate - no byte-level corruption was missed in any trial.
- **False Positive Rate** Verification of authentic fragments is deterministic: the random challenge r is derived from the stored `fpcc` hashes via a deterministic oracle, so re-verifying the same valid fragment always returns `CONSISTENT`. The measured false positive rate across 500 fragment checks (100 distinct blocks $\times$ 5 fragments each) is exactly 0.0%.
- **Byzantine Fault Tolerance** The system tolerates up to $n - m = 2$ Byzantine faults with $n = 5$ servers and reconstruction threshold $m = 3$. As long as at least m fragments from honest servers are available, the client recovers the original data correctly, even if the remaining servers return arbitrarily corrupted bytes.

10.1 Spoofing

10.1.1 S.1: Client Impersonation

The current Veri-Store prototype does not implement client authentication. An attacker who learns or or guesses a block identifier can issue PUT, GET, and DELETE requests to any server, effectively acting as the original uploader. Because block identifiers carry no credential binding, the attacker can overwrite existing fragments, retrieve stored objects belonging to other users, or permanently delete data without authorization.

In a multi-user production environment, client authentication would be necessary, implemented in such a way that such an attack would not be possible.

10.1.2 S.2: Server Impersonation

The client does not verify server identity before transmitting fragments. An attacker who can route traffic to a controlled host at a configured server port will receive raw fragment data in plaintext, since the current implementation does not use TLS. If an attacker intercepts connects to at least m of the n servers, they collect enough fragments to reconstruct the full original object.

In its prototype phase, this is not addressed, as we are only connecting to `localhost` servers. In an online system, however, server verification would be necessary to prevent against this attack.

10.1.3 S.3: Fingerprint Source Forgery

The `fpcc` is returned by the server inside the GET response body and is trusted by the client without an independent out-of-band authentication channel. A malicious server can construct an `fpcc` that is cryptographically consistent with arbitrarily modified fragment data by computing SHA-256 hashes and GF(256) fingerprints over the corrupted content directly. If the client receives such a forged `fpcc`, `Verifier.check()` will report `CONSISTENT` for the fraudulent fragments, bypassing Byzantine detection entirely.

Data would not be reconstructed with a single such fragment, as `client.py` would only accumulate fragments with the same "valid" `fpcc` - if an attacker modifies $< m$ fragments (1 or 2 in our implementation), then reconstruction would be halted and a `RetrievalError` raised. However, if m or greater servers collude with the same forged `fpcc`, reconstruction would succeed with the corrupted data.

This can be addressed by the client retaining an original copy of the `fpcc` and comparing it against incoming `fpcc`s. This won't mitigate the attack entirely (e.g., an adversary could still corrupt data), but they would no longer be able to do so silently.

10.2 Tampering

10.2.1 T.1: Fragment Corruption in Storage

A compromised storage server can modify the bytes of a stored fragment file between PUT and GET operations. When the client retrieves the fragment, `Verifier.check()` computes SHA-256 over the returned bytes and compares it against the hash recorded in the `fpcc`. Any modification, including a single bit flip, produces a mismatch and the fragment is silently discarded. The system fully mitigates this threat for up to $n - m = 2$ Byzantine servers; if more than two servers are compromised, the client cannot assemble m valid fragments and reconstruction fails.

10.2.2 T.2: Fragment Modification in Transit

An on-path network attacker can intercept HTTP traffic and modify fragment bytes or `fpcc` metadata in transit between the server and the client. Modifications to fragment bytes are caught by `Verifier.check()` (SHA-256 mismatch), but because the current implementation does not use TLS, there is no transport-layer protection for the `fpcc` itself. An attacker who modifies the `fpcc` in transit simultaneously with the fragment bytes can craft a consistent forgery (see T.3 below).

When done to a single fragment, while the `fpcc` and the underlying data would be consistent, the cross-server consistency check would fail and the modified fragment discarded.

However, if an adversary is able to simultaneously modify at least m fragments during transit, providing consistent fragment changes and `fpcc`s, the corrupted data will be accepted and used in construction.

10.2.3 T.3: Fingerprint Tampering

An attacker who can simultaneously modify a fragment's bytes and its corresponding `fpcc` entries can craft a consistent (corrupted fragment, forged `fpcc`) pair. The random challenge r is used in the GF(256)

fingerprint check is derived deterministically from the `fpcc` hashes via a public oracle. An attacker who controls the `fpcc` content can therefore compute r and produce a valid fingerprint for arbitrary data.

This attack succeeds only if the attacker can deliver both the corrupted fragment and the matching forged `fpcc` to the client – a condition enabled by the absence of server identity verification (section S.2, above) and unauthenticated `fpcc` transport (section S.3, above).

10.2.4 T.4: Erasure Code Parameter Manipulation

Fragment metadata returned in `GET` responses – including `total_n`, `threshold_m`, and `original_length` – is accepted by the client without independent verification against an out-of-band trusted record. These fields are distinct from `fpcc_json` and are not covered by the cross-server consistency check, so a server can manipulate them while returning an honest `fpcc_json` that passes fragment verification. The impact depends on which field is targeted.

`decoder.py` reads `total_n` and `threshold_m` from the first accepted fragment and then validates that all remaining accepted fragments agree. A single fragment with mismatched parameters therefore raises a `ValueError`, halting reconstruction – a denial-of-service effect achievable by one Byzantine server. For silent corruption of `total_n` or `threshold_m`, the attacker must control at least m servers so that all m accepted fragments carry the same forged value, satisfying the consistency check while driving the decoder to an incorrect configuration.

`original_length` is read from the first accepted fragment but is never cross-checked against the other $m - 1$ fragments. A single server whose response arrives first can therefore silently truncate or pad the reconstructed output without triggering a `ValueError`.

10.3 Repudiation

10.3.1 R.1: Anonymous Data Storage

An unauthenticated actor can submit arbitrary blocks without a durable identity binding. If abuse occurs (for example, storing prohibited content or intentionally filling capacity), operators cannot reliably attribute actions to a specific principal, weakening accountability and incident response.

10.3.2 R.2: Fragment Deletion Without Logging

A compromised or misconfigured storage server can delete local fragments and return standard HTTP error responses later, while leaving little or no tamper-evident audit trail. In that case, the system can observe data loss but cannot prove which server performed the deletion or when the deletion occurred.

10.3.3 R.3: Untrackable Verification Failures

When the client receives inconsistent fragment responses, weak request correlation and insufficiently structured logs can make it difficult to attribute corruption to a specific server instance over time. This enables a Byzantine server to intermittently misbehave while plausibly denying responsibility.

10.4 Information Disclosure

10.4.1 I.1: Fragment Eavesdropping

An on-path attacker observing client-to-server traffic can capture fragment payloads in transit. If the attacker collects at least m fragments for the same block, they can reconstruct the original plaintext object because erasure coding provides redundancy, not confidentiality.

10.4.2 I.2: Storage Server Data Access

A storage server operator (or a host compromise) can read fragment files at rest directly from disk. Although one server typically holds only one fragment per block, repeated observation across many blocks still discloses user data patterns and potentially sensitive content.

10.4.3 I.3: Fingerprint Analysis

An attacker who collects cross-checksum material (hashes, fingerprints, and metadata) can run offline analysis to correlate uploads, infer duplicate content patterns, or support dictionary style guessing against predictable objects. While fingerprints are not raw data, metadata leakage still increases attacker knowledge about stored content.

10.4.4 I.4: Error Message Information Leakage

Verbose API errors and debug logs may expose internal identifiers, verification outcomes, fragment indices, and topology details (server IDs and ports). This information can be used by attackers to target specific nodes, optimize denial-of-service behavior, or refine tampering attempts.

10.5 Denial of Service

10.5.1 D.1: Storage Exhaustion

10.5.2 D.2: Fragment Unavailability

10.5.3 D.3: Verification Flooding

10.5.4 D.4: Network Bandwidth Exhaustion

10.6 Elevation of Privilege

10.6.1 E.1: Code Injection via Malformed Input

10.6.2 E.2: Verification Bypass

10.6.3 E.3: Cryptographic Parameter Override

10.6.4 E.4: Server-to-Server Privilege Escalation

11 Threat Resolutions

11.1 Spoofing Threat Resolutions

11.1.1 S.1: Client Impersonation

- **Status**: Requires mitigation for multi-user deployment. Partially mitigated by current design.
- **Resolution**: The current implementation requires a bearer token on all `PUT`, `GET`, and `DELETE` endpoints, which blocks unauthenticated clients from impersonating a legitimate caller. The FastAPI server refuses to start without a configured `VERI_STORE_TOKEN` and each route verifies the supplied bearer token before any fragment operation is performed.

  For the current single-client implementation, this token is shared across the deployment rather than bound to an individual user identity, so any party that learns the token can still act as the client and access any known block identifier. A production deployment would therefore need to extend this design with per-user authentication and authorization and enforcing ownership checks on endpoint calls.
- **Owner**: Mac
- **Testing**: Integration tests should verify that `PUT`, `GET`, and `DELETE` requests with an invalid or missing bearer token return `HTTP 401`, while the `/health` endpoint remains publicly accessible. Configuration tests should verify that the server fails to initialize when `VERI_STORE_TOKEN` is not set.

  For a future multiple-user implementation, authorization tests should be added to confirm that one authenticated user cannot retrieve, overwrite, or delete a fragment belonging to a different authenticated user.

11.1.2 S.2: Server Impersonation

- **Status**: Accepted risk for current implementation. Would require mitigation for any networked production deployment.
- **Resolution**: In the current implementation, the client communicates with the servers over HTTP in the clear using configured host and port values and does not verify a server certificate, pinned key, or other server identity credentials before sending or accepting fragment data. As a result, bearer token authentication protects the server from unauthenticated clients, but it does not protect the client from sending fragments to an adversarial endpoint that successfully impersonate a storage server.

  This is considered an acceptable risk in this prototype since the deployment is limited to trusted `localhost` processes on a single machine. For any non-local or multi-host deployment, however, VeriStore should run behind TLS with certificate validation and infrastructure-level server authentication, so that the client can confirm it is communicating with the intended storage node before transmitting fragment data or trusting retrieval responses.
- **Owner**: Mac
- **Testing**: For this prototype, manually verify that only `localhost` endpoints are targeted and does not claim transport security beyond that environment.

For a production deployment, integration tests should confirm that the client rejects connections with invalid, expired, or mismatched server certificates, and accepts connections only when the certificate chain and expected server identity are successfully validated.

11.1.3 S.3: Fingerprint Source Forgery

- **Status**: Requires mitigation
- **Resolution**: During retrieval, the client currently trusts the first successful `fpcc_json` it receives and uses it as the reference value for subsequent fragment verification. This means the client verifies the fragment consistency against a server-supplied `fpcc`, not against a separately trusted record. A byzantine server, or a set of colluding byzantine servers, can therefore construct a forged `fpcc` that is internally consistent with corrupted fragments. If at least m servers return the same forged `fpcc`, the client may accept those fragments and reconstruct corrupted data without detecting the deception.

  The mitigation is to establish an authenticated source of truth for the `fpcc`. For example, the client should persist the original `fpcc` produced at `PUT` time and compare all retrieved fragments against the retained value, or the system should store a signed MAC-protected `fpcc` in a trusted metadata service.

  Cross-server agreement alone is not sufficient, because multiple malicious servers can agree on the same forged metadata.
- **Owner**: Mac
- **Testing**: Add retrieval tests in which one server returns a forged `fpcc` and corrupted fragment, confirming the client rejects it when compared against a trusted stored `fpcc`. Add a stronger adversarial test in which m colluding servers return the same forged `fpcc` and mutually consistent corrupted fragments and verify that retrieval still fails once the client uses the trusted original `fpcc` rather than a server-supplied one. Regression tests should also verify that honest retrieval still succeeds when all servers return the correct `fpcc`.

11.2 Tampering Threat Resolutions

11.2.1 T.1: Fragment Corruption in Storage

- **Status**: Mitigated in current implementation
- **Resolution**: Veri-Store detects byte-level corruption of stored fragments during verification and retrieval. Each fragment is checked against the stored `fpcc` using a `SHA-256` hash check, and for indices $i < m$, an additional homomorphic fingerprint check.

  A storage server can still alter bytes on disk, but the modified fragment will not verify against the original `fpcc` and will therefore be rejected before reconstruction. In the current 3-of-5 deployment, the system can tolerate such tampering as long as at least $m = 3$ honest fragments remain available.
- **Owner**: Mac
- **Testing**: Use the existing verifier and client retrieval tests that inject corrupted fragment bytes and confirm that verification returns `HASH_MISMATCH` or otherwise rejects the tampered fragment.

  End-to-end tests should also confirm that retrieval succeeds when fewer than $n - m = 2$ fragments are corrupted and fails cleanly when too many fragments are corrupted.

11.2.2 T.2: Fragment Modification in Transit

- **Status**: Partially mitigated in current implementation
- **Resolution**: If an attacker modifies fragment bytes while they are in transit, the modified payload will usually be detected by the `fpcc`-based verification used for storage integrity, described in section T.1. The receiving server checks uploaded fragments during `PUT`, and the client re-verifies fragments during `GET` before reconstruction.

  As a result, byte-level tampering of fragment payloads does not normally succeed silently. However, this mitigation applies only when the fragment is checked against a trusted `fpcc`. If the attacker can also tamper with the `fpcc` or impersonate a server, the attack becomes part of the threats described in sections T.3 or S.2 rather than a simple fragment-byte modification. For that reason, in a production deployment, transport hardening such as TLS is recommended.
- **Owner**: Mac
- **Testing**: Verify that tampering with fragment bytes in `PUT` requests causes the server to reject the fragment with `HTTP 422`, and that tampering with bytes in `GET` responses causes the client to discard the fragments during retrieval. For deployment hardening, add integration tests confirming that transport protections prevent or survive in-transit modification attempts.

11.2.3 T.3: Fingerprint Tampering

- **Status**: Requires mitigation
- **Resolution**: The current verification logic proves that a fragment is consistent with the `fpcc` supplied to the checker, but it does not independently authenticate the source of that `fpcc`. An attacker who can alter both fragment data and the associated `fpcc` can construct a forged pair that remains internally consistent. During retrieval, the client currently adopts the first successful `fpcc` it receives as the reference value, so a malicious server or colluding set of servers can potentially cause corrupted fragments to be accepted if they all present the same forged `fpcc`.

  This can be mitigated by binding retrieval to a trusted `fpcc`, such as by storing the original client-generated `fpcc` locally, retrieving it from a trusted metadata service, or authenticating it with a digital signature or MAC. Cross-server agreement alone is not sufficient, because multiple malicious servers can agree on the same forged metadata.

  See also section S.3.
- **Owner**: Mac
- **Testing**: Add tests in which a server returns a forged `fpcc` together with tampered fragment data, and confirm that the client rejects the response when compared against a trusted stored `fpcc`. A stronger test will consist of m colluding malicious servers that all return the same forged `fpcc` – retrieval should still fail once the client verifies against the trusted original `fpcc` rather than a server-supplied one.

11.2.4 T.4: Erasure Code Parameter Manipulation

- **Status**: Partially mitigated, but requires additional mitigation

- **Resolution**: The current implementation already validates some coding-parameter tampering, but not all of it. On `PUT`, malformed requests with impossible parameters such as `threshold_m > total_n` are rejected by request validation, and duplicate writes with conflicting metadata are rejected with `HTTP 409`. During decode, the client also checks that all accepted fragments agree on `block_id`, `total_n`, and `threshold_m`. These controls limit straightforward manipulation of coding parameters.

  However, the decoder currently takes `original_length` from the first accepted fragment and does not cross-check it against the other verified fragments. As a result, a malicious server whose response is accepted first can potentially cause silent truncation or padding effects in the reconstructed output.

  The mitigation is to authenticate all reconstruction metadata as part of a trusted object manifest or bind it cryptographically to the trusted `fpcc`, and to require the client to confirm that all accepted fragments agree on `original_length` before decoding.

- **Owner**: Mac

- **Testing**: The existing validation tests reject impossible `PUT` requests (e.g., `threshold_m > total_n`).

  Add retrieval tests where one fragment reports manipulated `total_n`, `threshold_m`, and/or `original_length`. The client should fail closed on metadata disagreement, and after the mitigation is implemented, tests should verify that a forged `original_length` can no longer silently truncate or extend reconstructed output.

11.3 Repudiation Threat Resolutions

11.3.1 R.1: Anonymous Data Storage

- **Status**: Accepted risk for academic prototype

- **Resolution**: The current implementation does not enforce authentication for HTTP endpoints, prioritizing simplicity for the course project scope. Production deployments would add an authentication layer (token-based or certificate-based) at the reverse proxy or API gateway to bind actions to authenticated principals. For the controlled laboratory environment, anonymous access is acceptable.

- **Owner**: Albert

- **Testing**: Manual verification that endpoints accept requests without authentication headers. For production systems, integration tests should verify that unauthenticated requests are rejected with HTTP 401 when authentication is enabled.

11.3.2 R.2: Fragment Deletion Without Logging

- **Status**: Already mitigated by request logging implementation

- **Resolution**: The FastAPI middleware logs all HTTP requests including DELETE operations in `src/network/server.py`, providing an audit trail with timestamps, server identity, fragment identifiers, and outcomes. Logs are written to standard output and can be aggregated to centralized log management systems for long-term retention.

- **Owner**: Albert

- **Testing**: Perform DELETE requests and verify that log messages appear for each operation with correct block_id, index, and status code. Test suite should capture log output and assert that deletion events are properly recorded.

11.3.3 R.3: Untrackable Verification Failures

- **Status**: Already mitigated by verification failure logging
- **Resolution**: When fragment verification fails during PUT operations, the server logs detailed context at WARNING level in `src/network/server.py`, recording server ID, block ID, fragment index, and the specific check that failed. This structured logging enables operators to correlate failures over time and attribute Byzantine behavior by analyzing aggregated logs for patterns of repeated failures from specific server instances.
- **Owner**: Albert
- **Testing**: Inject corrupted fragments during PUT operations and verify that WARNING level log entries appear with full context. Test multiple corrupted requests and verify that log analysis can identify patterns of failures from a specific server.

11.4 Information Disclosure Threat Resolutions

11.4.1 I.1: Fragment Eavesdropping

- **Status**: Dependency responsibility
- **Resolution**: Transport layer confidentiality is outside the scope of veri-store's erasure coding and integrity verification design. In production deployments, operators must terminate TLS at the edge (reverse proxy, load balancer, or API gateway) to provide transport encryption. The erasure coding scheme provides no inherent confidentiality; applications requiring confidentiality must encrypt data at the application layer before erasure encoding or rely on TLS for transport security.
- **Owner**: Albert (deployment/documentation)
- **Testing**: Verify that production deployments use HTTPS endpoints. Network traffic capture tools should confirm that fragment payloads are encrypted in transit. For the academic prototype, document the assumption that HTTP is acceptable in controlled laboratory networks.

11.4.2 I.2: Storage Server Data Access

- **Status**: Accepted risk with operational controls
- **Resolution**: Storage servers have direct filesystem access to fragment data at rest in the `data/server_N` directories. A single compromised server can read its local fragments but cannot reconstruct complete objects without collecting at least m fragments from other servers. In production, operators should use standard operating system access controls to restrict filesystem access to the server process user account. For stronger confidentiality, applications can encrypt data before submitting it to veri-store, treating the storage system as untrusted infrastructure.

- **Owner**: Albert
- **Testing**: Review filesystem permissions on fragment storage directories. Verify that only the server process owner can read fragment files. For encrypted deployments, verify that stored fragments contain ciphertext rather than plaintext by examining on-disk data.

11.4.3 I.3: Fingerprint Analysis

- **Status**: Partially mitigated by cryptographic hash collision resistance
- **Resolution**: The fingerprinted cross-checksum includes SHA-256 hashes of all fragments and homomorphic fingerprints for the first m fragments. While these values do not directly expose plaintext content, they enable offline correlation attacks where an attacker can identify duplicate uploads by matching hash sets. The SHA-256 hash function provides preimage resistance, making dictionary attacks computationally expensive but not impossible for low-entropy data. Production systems requiring stronger privacy guarantees should encrypt objects before dispersal or add randomness to block identifiers to decorrelate metadata across uploads.
- **Owner**: Albert
- **Testing**: Store the same object multiple times and verify that fpcc values are deterministic. Attempt to correlate uploads by comparing stored fpcc_json values. For systems requiring privacy, verify that client-side encryption is applied before erasure encoding.

11.4.4 I.4: Error Message Information Leakage

- **Status**: Already mitigated by structured error responses and controlled logging
- **Resolution**: HTTP error responses return minimal context to clients via FastAPI's HTTPException mechanism in `src/network/server.py`, including only block ID, fragment index, and generic status descriptions without disclosing internal server topology or storage paths. Detailed diagnostic information such as verification failure details, stack traces, and internal state is written to server-side logs rather than returned to clients. The logging implementation records comprehensive context for operator debugging while preventing information disclosure to potentially hostile clients.
- **Owner**: Albert
- **Testing**: Trigger error conditions (404 not found, 409 conflict, 422 validation failure) and verify that HTTP responses contain only generic error messages without internal details. Verify that detailed diagnostic information appears in server logs but not in client responses. Test that stack traces and filesystem paths are not leaked in production error responses.

11.5 Denial of Service Threat Resolutions

11.5.1 D.1: Storage Exhaustion

- **Status**:
- **Resolution**:

- **Owner**:
- **Testing**:

11.5.2 D.2: Fragment Unavailability

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5.3 D.3: Verification Flooding

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5.4 D.4: Network Bandwidth Exhaustion

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6 Elevation of Privilege Threat Resolutions

11.6.1 E.1: Code Injection via Malformed Input

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6.2 E.2: Verification Bypass

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6.3 E.3: Cryptographic Parameter Override

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6.4 E.4: Server-to-Server Privilege Escalation

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

12 External Security Notes

12.1 Deployment Recommendations

We recommend a simple but safe deployment setup:

- Keep the five storage servers on a private network and do not expose them directly to the public internet.
- Use HTTPS through a reverse proxy if clients are not running on localhost.
- Set a strong shared API token and rotate it if it is leaked.
- Run each server process with limited filesystem permissions so it can only access its own data folder.
- Turn on request logging so failed verification events are easy to trace during debugging and demos.

12.2 Known Limitations

Veri-Store is primarily an *integrity* prototype, not a complete secure storage system. Its verification logic is effective at detecting fragment corruption and many forms of byzantine tampering, but several important security properties remain outside the current design.

- **No confidentiality guarantees.** Fragments are stored and transmitted as plaintext unless the integrating application adds encryption or deploys the system behind TLS. Erasure coding alone does not make the data confidential.
- **The client does not retrieve against a separately trusted FPCC.** During `GET`, the client currently adopts the first successfully returned `fpcc_json` as the reference value for fragment verification. This means the client proves that fragments are internally consistent with that `fpcc`, but not that the `fpcc` itself came from a trusted source. If at least m malicious servers return the same forged `fpcc` and mutually consistent fragments, the client may reconstruct attacker-chosen data.
- **Block identity is not cryptographically bound into verification.** The current `fpcc` commits to fragment hashes and fingerprints, but not to the application-level `block_id`. As shown by the penetration-testing results (`.\docs\experiments\pentest-attack-matrix.md`), a valid fragment or a full stale response can be replayed under a different block namespace and still pass the low-level fragment checks. This is an accepted limitation of the current prototype and would need to be addressed before claiming stronger object-authenticity guarantees.
- **Authentication is deployment-wide rather than user-specific.** The bearer token protects the service from unauthenticated access, but any party that learns the shared token can read, write, or delete any block whose identifier it knows. There is no per-user authorization, ownership check, or audit identity binding in the current implementation. This is an accepted risk, as the current implementation is meant for a single user, but would need to be mitigated for a multi-user deployment.
- **Security depends on threshold assumptions.** The integrity story assumes that fewer than m servers collude maliciously when the client is relying on server-supplied metadata. The system also assumes the cryptographic primitives behave as expected, in particular the collision resistance of

SHA-256 and the random-oracle-style behavior used in the fingerprint construction. The collision resistance is empirically demonstrated in `.\docs\experiments\collision-probability.md`.

- **Availability and denial-of-service resistance are limited.** Veri-Store can tolerate missing or corrupted fragments up to the reconstruction threshold, but it does not currently provide strong rate limiting (it does have basic rate limiting), storage quotas, admission control, or recovery orchestration suitable for adversarial production environments.

These limitations do not invalidate the prototype's main goal, which is to demonstrate verified integrity checking for erasure-coded fragments. They do, however, define the boundary of the security claims that this document can make: Veri-Store currently offers useful corruption detection and threshold-based integrity checks, but not full end-to-end authenticity, confidentiality, or production-grade access control.

12.3 Integration Considerations

When Veri-Store is embedded into a larger application, it should be treated as a specialized *integrity-preserving fragment store*, not as a complete secure storage platform. The integrating system must therefore be explicit about which security properties are provided by Veri-Store itself and which are still the responsibility of the surrounding application or deployment environment.

- **What Veri-Store provides.** Veri-Store provides threshold-based fragment storage, deterministic fragment verification against an `fpcc`, and rejection of many corrupted or malformed fragment responses. In the normal honest-client workflow, it helps ensure that reconstructed data has not been silently modified by a single faulty or byzantine storage node.
- **What the integrating system must provide.** The surrounding system must still provide end-user authentication, authorization, object naming policy, confidentiality, key management if encryption is used, transport security, deployment hardening, and operational monitoring. Veri-Store should be viewed as one component in a broader security architecture rather than as the architecture itself.
- **Trusted metadata handling.** If the application needs stronger authenticity guarantees than the current prototype provides, it should retain the original client-generated `fpcc` in a trusted location and use that value during retrieval, rather than trusting the first server-supplied `fpcc`. A production integration may also bind the `fpcc` to an authenticated object manifest, digital signature, or MAC-protected metadata service.
- **Confidentiality layering.** Applications that store sensitive data should encrypt objects before passing them to Veri-Store. In that model, Veri-Store protects the integrity and recoverability of ciphertext fragments, while confidentiality would need to be enforced by an external encryption layer.
- **Identifier semantics.** The current implementation treats `block_id` as an application-level routing key rather than a cryptographically authenticated part of the fragment commitment. Integrating systems should therefore generate block identifiers carefully, avoid exposing predictable identifiers unnecessarily, and avoid assuming that block identity is currently protected to the same degree as fragment contents.
- **Failure handling.** The integrating system should treat retrieval errors, verification failures, and repeated byzantine detections as security-significant events rather than ordinary application errors. These events should be surfaced to logs, monitoring, and incident-response workflows instead of being silently retried without investigation.

In short, Veri-Store fits best as a lower-layer integrity service. It can strengthen a larger system's resilience to fragment corruption and some classes of byzantine storage behavior, but it relies on the integrating system to supply the rest of the security envelope.

13 Verification and Testing Plan

13.1 Security Test Suite

13.2 Code Review Process

In review, we check four things:

- The change does not weaken integrity checks or bypass verification paths.
- Error handling is safe and returns clear HTTP status codes.
- Input validation is still strict for API payloads.
- Tests were added or updated for the changed behavior.

13.3 Regression Testing

We keep a small set of repeatable security checks:

- Byzantine fragment detection still rejects corrupted data.
- Retrieval still succeeds with only m healthy servers.
- Edge-case payload tests still pass, including binary and large inputs.

14 Document Maintenance

14.1 Review Schedule

Security model reviews follow a predictable schedule tied to engineering milestones:

- Weekly during active development (Sunday code review window).
- At the end of each project week before TODO ownership is updated.
- Before demos or external presentations where security claims are shown.
- Immediately after changes to security-critical code paths, including:
  - `src/network/server.py` request validation and verification handling.
  - `src/network/client.py` retrieval verification and quorum behavior.
  - `src/verification/verifier.py` consistency checks and decision logic.

Albert owns the maintenance schedule for backend-owned sections, Mac owns the same process for security-owned sections, and Section 14 status is confirmed jointly during weekly sync.

14.2 Change Process

All updates to this document use the same workflow as code changes:

1. Identify trigger: a new threat, mitigation change, dependency update, or behavior change in implementation.
2. Update the relevant section in `.\docs\security-model\sections\` with concrete impact and file/function references when available.
3. Run peer review with the section owner pair (Albert/Mac) during the next scheduled review, or immediately for high-risk changes.
4. Merge the documentation update in the same PR as the code change when possible; otherwise link the follow-up documentation PR in the code PR notes.
5. Record the update in Section 14.3 (Version History) with date and summary.

If an implementation change affects a documented mitigation but the document is not updated in the same cycle, the change is considered incomplete until the security model is synchronized.

14.3 Version History

- **Version 0.1** (February 16, 2026): Initial draft - high-level security model with component stubs

- **Version 0.1.001** (February 16, 2026):
  - Added descriptions of external libraries
  - Added Administrator trust level and example administrators
- **Version 0.1.002** (March 1, 2026):
  - Split `.tex` file into sections for easier maintenance.
  - Changes to preamble re: formatting and spacing.
- **Version 0.1.1** (March 15, 2026):
  - Completed sections 1.1 and 1.2.
  - Inserted a `\newpage` control at the end of each section document.
  - Completed section 3
  - Completed section 6.2 & 6.4
  - Updated and verified section 4.1
- **Version 0.1.2** (March 28, 2026):
  - Completed Section 8, "Protected Assets"
- **Version 0.2.0** (April 4, 2026):
  - Added Section 10 intro, "Security Guarantees Achieved"
  - Added Section 10.1, "Spoofing"
  - Added Section 10.2, "Tampering"